

Universidad Nacional Autónoma de México



Facultad de Ingeniería

División de Ingeniería Eléctrica



Laboratorio Computación Gráfica e Interacción
Humano-Computadora (6590)

Práctica 3: Modelado jerárquico

Nombre: Calderón Gutiérrez Victor Emiliano

No. Cuenta: 423097331

Grupo (laboratorio): 03

Grupo (teoría): 02

Semestre 2026-1

Fecha de entrega: 21/09/2025

Calificación: _____

1.Ejercicios

Para esta práctica se terminó de crear el código para la grúa y también se creó un nuevo código sobre un gato. Vamos por partes, primero revisando el código de la grúa. Se metieron varias modificaciones.

La modificación más importante es que se pedía que se hiciera el movimiento de las llantas independiente, para eso me metí al código de Window.h en donde agregué las variables para trabajar con nuevas articulaciones.

```
23     GLfloat getarticulacion1() { return articulacion1; }
24     GLfloat getarticulacion2() { return articulacion2; }
25     GLfloat getarticulacion3() { return articulacion3; }
26     GLfloat getarticulacion4() { return articulacion4; }
27     GLfloat getarticulacion5() { return articulacion5; }
28     GLfloat getarticulacion6() { return articulacion6; }
29     GLfloat getarticulacion7() { return articulacion7; }
30     GLfloat getarticulacion8() { return articulacion8; }
31     GLfloat getarticulacion9() { return articulacion9; }
32
33     ~Window();
34 private:
35     GLFWwindow *mainWindow;
36     GLint width, height;
37     GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6, articulacion7, articulacion8, articulacion9;
38     bool keys[1024];
```

Una vez que se tuvieron creadas las variables también modifiqué el código en Window.cpp en donde ahora si agregué la lógica e hice que las rotaciones de las nuevas articulaciones se hicieran con B, N y M.

```
154
155     if (key == GLFW_KEY_B)
156     {
157         theWindow->articulacion7 += 10.0;
158     }
159     if (key == GLFW_KEY_N)
160     {
161         theWindow->articulacion8 += 10.0;
162     }
163     if (key == GLFW_KEY_M)
164     {
165         theWindow->articulacion9 += 10.0;
166     }
167
```

Una vez que se tuvo toda la base, se procedió a crear el código para generar la grúa. De las principales cosas y considerando la jerarquía se procedió a crear otras variables auxiliares para el modelo, esto para poder considerar donde va cada llanta y la base.

```

glm::mat4 model(1.0); //Inicializar matriz de Modelo 4x4
glm::mat4 modelaux(1.0); //Inicializar matriz de Modelo 4x4
glm::mat4 base(1.0);
glm::mat4 llanta1(1.0);
glm::mat4 llanta2(1.0);
glm::mat4 llanta3(1.0);
glm::mat4 llanta4(1.0);
glm::mat4 base1(1.0);

```

Adicional a eso se continuó con la creación de estos objetos en el código:

```

// Cuerpo de las llantas
base = glm::translate(base, glm::vec3(0.0f, -3.0f, 0.0f));
llanta1 = llanta2 = llanta3 = llanta4 = base;
base = glm::scale(base, glm::vec3(6.0f, 1.5f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(base));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMesh();

// Base
base = glm::translate(base, glm::vec3(0.0f, 1.0f, 0.0f));
base1 = glm::scale(base, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(base1));
meshList[4] -> RenderMeshGeometry();

// Articulación llanta 1
model = llanta1;
model = glm::translate(model, glm::vec3(2.0f, 0.0f, 2.1f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
llanta1 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

```

```

// Llanta 1
llanta1 = glm::scale(llanta1, glm::vec3(2.5f, 2.5f, 2.5f));
llanta1 = glm::rotate(llanta1, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(llanta1));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();

// Articulación llanta 2
model = llanta2;
model = glm::translate(model, glm::vec3(-2.0f, 0.0f, 2.1f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
llanta2 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// Llanta 2
llanta2 = glm::scale(llanta2, glm::vec3(2.5f, 2.5f, 2.5f));
llanta2 = glm::rotate(llanta2, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(llanta2));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();

```

```

// Articulación llanta 3
model = llanta3;
model = glm::translate(model, glm::vec3(2.0f, 0.0f, -2.1f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
llanta3 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// Llanta 3
llanta3 = glm::scale(llanta3, glm::vec3(2.5f, 2.5f, 2.5f));
llanta3 = glm::rotate(llanta3, glm::radians(270.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(llanta3));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();

// Articulación llanta 4
model = llanta4;
model = glm::translate(model, glm::vec3(-2.0f, 0.0f, -2.1f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
llanta4 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

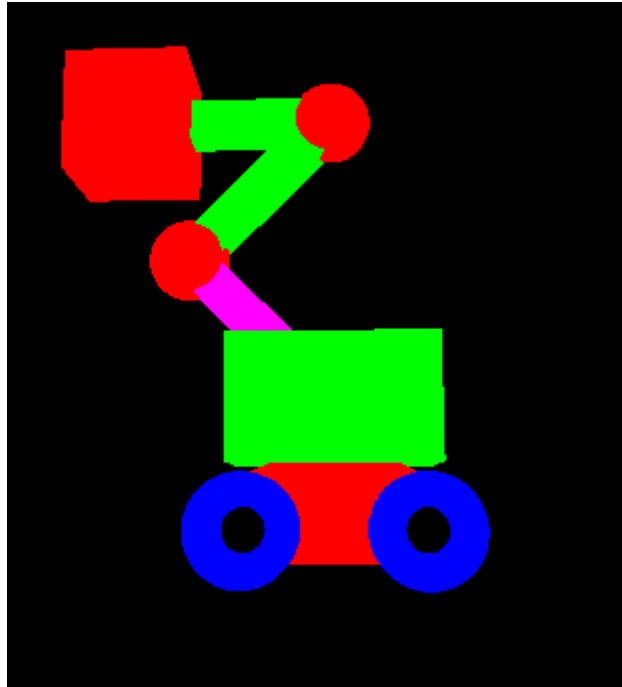
```

```

// Llanta 4
llanta4 = glm::scale(llanta4, glm::vec3(2.5f, 2.5f, 2.5f));
llanta4 = glm::rotate(llanta4, glm::radians(270.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(llanta4));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();

```

Todo ese código me permitió generar lo que faltaba y al correr el programa la grúa se ve de la siguiente manera:



Debido a que el movimiento de las llantas es casi imperceptible en fotos se invita a correrlo de forma local, pero revisando el código creado creo que es fácil identificar que si se implementa el movimiento de cada articulación por separado.

Ahora, la segunda parte fue la creación de un animal, en este caso el que decidí hacer es un gato. Para esto se mantuvieron las modificaciones de las articulaciones hechas en Window.cpp y Window.h. Pero las modificaciones diferentes son las siguientes. Primero igual se crearon una gran cantidad de variables para ir guardando el modelo para usar la jerarquía.

```
glm::mat4 model(1.0); //Inicializar matriz de Modelo 4x4
glm::mat4 modelaux(1.0); //Inicializar matriz de Modelo 4x4
glm::mat4 cuerpo(1.0);
glm::mat4 cabeza(1.0);
glm::mat4 pata1(1.0);
glm::mat4 pata2(1.0);
glm::mat4 pata3(1.0);
glm::mat4 pata4(1.0);
glm::mat4 cola(1.0);
```

Ahora, para hacer todo lo demás se siguió haciendo uso de las jerarquías y se decidieron hacer dos modificaciones, usar solo dos colores para el modelo. Se usó gris como el color de mi gato y color rojo para identificar las articulaciones.

```
glm::vec3 colorGato = glm::vec3(0.0f, 0.0f, 0.0f); //Color para el gato
glm::vec3 colorArticulaciones = glm::vec3(1.0f, 0.0f, 0.0f); // Color rojo para las articulaciones
```

Por último, se hizo uso de la jerarquía para presentar los objetos, también de las técnicas usadas en prácticas pasadas.

```

// Cuerpo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f));
cabeza = cuerpo = model; // Se guarda la posición de la cabina para generar la base
model = glm::scale(model, glm::vec3(5.0f, 1.5f, 2.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
meshList[0]->RenderMesh();

// Articulación pata 1
model = cuerpo;
model = glm::translate(model, glm::vec3(2.0f, -0.75f, 0.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
pata1 = model; // Se guarda la posición de la pata 1 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

// Primera parte pata 1
model = pata1; // Se usa la matriz pata1 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.75f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

```

```

// Segunda articulación pata 1
model = pata1; // Se usa la matriz pata1 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
pata1 = model; // Se guarda la posición de la pata 1 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

// Tercer parte pata 1
model = pata1; // Se usa la matriz pata1 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.75f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

// Articulación pata 2
model = cuerpo;
model = glm::translate(model, glm::vec3(-2.0f, -0.75f, 0.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
pata2 = model; // Se guarda la posición de la pata 2 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

```

```

// Primera parte pata 2
model = pata2; // Se usa la matriz pata2 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.75f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

// Segunda articulación pata 2
model = pata2; // Se usa la matriz pata2 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
pata2 = model; // Se guarda la posición de la pata 2 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

// Tercer parte pata 2
model = pata2; // Se usa la matriz pata2 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.75f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

```

```

// Articulación pata 3
model = cuerpo;
model = glm::translate(model, glm::vec3(2.0f, -0.75f, -0.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
pata3 = model; // Se guarda la posición de la pata 3 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

// Primera parte pata 3
model = pata3; // Se usa la matriz pata3 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.75f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

// Segunda articulación pata 3
model = pata3; // Se usa la matriz pata3 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
pata3 = model; // Se guarda la posición de la pata 3 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

```



```

// Tercer parte pata 3
model = pata3; // Se usa la matriz pata3 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.75f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

// Articulación pata 4
model = cuerpo;
model = glm::translate(model, glm::vec3(-2.0f, -0.75f, -0.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
pata4 = model; // Se guarda la posición de la pata 4 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

// Primera parte pata 4
model = pata4; // Se usa la matriz pata4 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.75f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

```

```

// Segunda articulación pata 4
model = pata4; // Se usa la matriz pata4 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
pata4 = model; // Se guarda la posición de la pata 4 para hacer la jerarquía con la pata
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

// Tercer parte pata 4
model = pata4; // Se usa la matriz pata4 para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.75f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[0]->RenderMesh();

// Cabeza
model = cabeza;
model = glm::translate(model, glm::vec3(3.0f, 1.75f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
cabeza = model; // Se guarda la posición de la cabeza para generar las orejas
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

```



```

// Oreja izquierda
model = cabeza; // Se usa la matriz cabeza para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.65f, 0.5f, -0.35f));
model = glm::scale(model, glm::vec3(1.15f, 1.15f, 1.15f));
model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

// Oreja derecha
model = cabeza; // Se usa la matriz cabeza para continuar la jerarquía
model = glm::translate(model, glm::vec3(0.65f, 0.5f, 0.9f));
model = glm::scale(model, glm::vec3(1.15f, 1.15f, 1.15f));
model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

// Articulación cola
model = cuerpo;
model = glm::translate(model, glm::vec3(-2.5f, 0.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(1.0f, 0.0f, 0.0f));
cola = model; // Se guarda la posición de la cola para hacer la jerarquía con la cola
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulaciones));
sp.render();

```

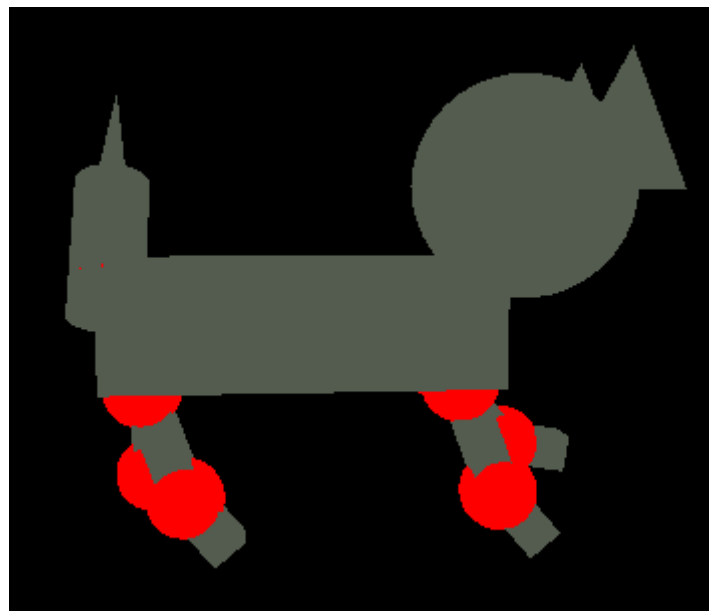
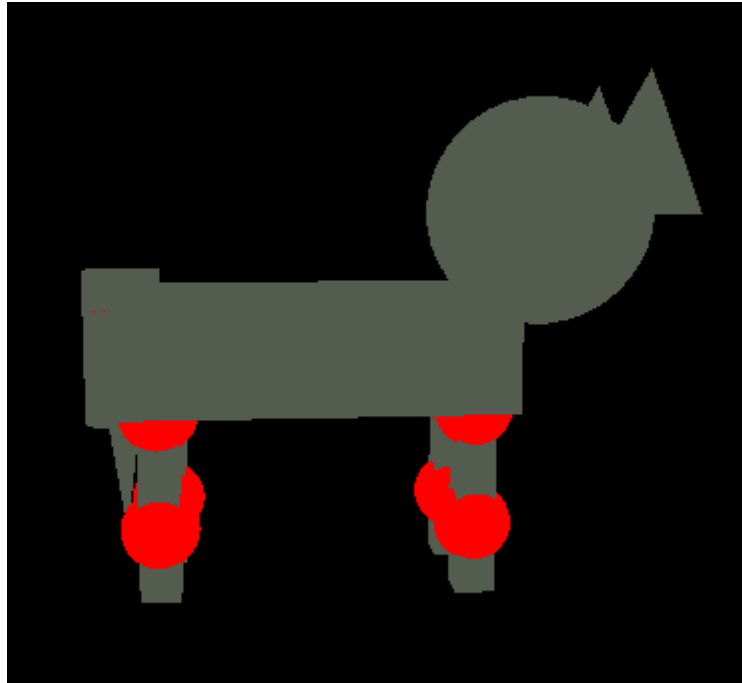
```

// Cola
model = cola; // Se usa la matriz cola para continuar la jerarquía
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
cola = model; // Se guarda la posición de la cola para hacer la jerarquía con la cola
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorGato));
meshList[2]->RenderMeshGeometry();

// Punta cola
model = cola;
model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[4]->RenderMeshGeometry();

```

Al final, una vez que se corre este programa se tiene que el gato se ve de la siguiente forma:



Debido a que aquí es más perceptible el movimiento se añade una segunda imagen para notar el desplazamiento en las articulaciones, las cuales se mueven de forma independiente.

2.Problemas

Ahora tuve un gran problema a la hora de hacer las rotaciones, de hecho, las pocas que llegué a hacer no fueron las mejores que se pudieran hacer. En la práctica anterior sentí que necesitaba fundamentos teóricos para saber como mover las formas, y de cierta forma sí, pero también con práctica sale, ahora creo que necesito prácticas más con rotaciones para poder lograr que en próximas prácticas pueda mejorar y que me queden mucho más claras.

3. Conclusión

El nivel en esta práctica ya se nota mucho más complejo, ya que se necesita entender en su totalidad todo lo que hemos hecho previamente. Afortunadamente no estoy tan perdido entonces, aunque requirió mucho tiempo, esta práctica no fue imposible, ya que se logró hacer todo, aunque hay muchas áreas de mejora. Gracias a esta práctica, ya quedó mucho más claro lo que es el modelo jerárquico, lo cual permite programar los modelos 3d de una forma mucho más sencilla.

4. Bibliografía

- Ing. Jose Roque RG. (2024, 29 febrero). Práctica 4 Modelado jerárquico [Vídeo].

YouTube. <https://www.youtube.com/watch?v=1jDotSbKfGo>